

模式 B Agent 开发与上传指南

模式 B · Cloudflare 容器

文档版本 1.0 · 更新日期 2026-09-28

范围说明：本指南只覆盖模式 B。平台托管独立节点流程请查看[模式 A 技术预览指南](#)，并留意其中列出的生产环境已知缺口。

官方入口：[Agent 开发中心](#) · [聊天补全 API](#)

文档目录

1. 开始前准备什么
2. 应用必须满足的运行规范
3. 模型接口与平台 Key 的接入方式
4. 页面与流式接口要注意什么
5. 构建镜像并在本机验证
6. 推送到阿里云公共镜像仓库
7. 在平台填写部署配置并发布
8. 更新、回滚和示例项目特别说明
9. 常见问题排查表
10. 发布前检查与可复制的开发需求

本指南适用于已有 Web 应用或准备使用 Python + Vue 开发 Agent，并希望通过平台发布给用户使用的开发者。你需要交付一个可运行的 Docker 镜像；平台负责后续镜像处理、运行时部署和用户实例启动。无需自行部署平台的 Cloudflare Worker。

当前已验证的交付路径是：本地开发 → 构建 amd64 镜像 → 推送阿里云仓库 → 平台填写外部镜像地址 → 校验、扫描与部署 → 自测 → 审核发布。

本页讲解平台托管 Agent 的部署。普通 API 客户端如何创建个人 Key，见[快速开始](#)；两种场景的凭据配置不同。

本文使用 `my-agent:1.0.0` 作为示例镜像名，`claude-opus-5` 作为示例模型。镜像名、仓库地址和模型需按你的实际项目替换。该模型名称是网关模型标识，是否可用由平台的模型列表、账号分组和渠道决定。

1. 开始前准备什么

准备项	你需要确认的内容
平台开发者账号	可以创建 Agent，编辑部署配置，并进行开发者自测
本地 Docker	Docker Desktop 或 Docker Engine 已启动，能够执行 <code>docker version</code>
项目源码	有 <code>Dockerfile</code> 、依赖文件、网页入口、后端接口和健康检查
阿里云镜像仓库	已创建仓库，并取得控制台提供的仓库地址、登录用户名和登录方式
可用模型	默认模型在平台允许范围内，也在该 Agent 的模型允许列表中

不要求开发者把个人模型 Key 写入应用。平台运行时会给每个“用户 × Agent”注入专属凭据。阿里云仓库登录密码、平台模型 Key、Cloudflare 管理凭据是不同用途的凭据，不能互换。

上传入口怎么选

入口或文件	本指南中的用途
平台“外部镜像仓库”	当前已验证可用的主流程；填写完整 Registry 镜像引用
推送阿里云仓库	使用 Docker 把镜像上传到自己的仓库，再让平台拉取
docker save 生成的 TAR	镜像备份、迁移或交给支持导入的一方；不代表平台当前有浏览器直传入口
源码 ZIP	交付源代码，不能作为容器镜像地址提交
平台直接 Docker Push / 浏览器 TAR 上传	在提供的规格基线中属于待补齐能力；只有平台明确上线后，才按该入口说明使用

下面的步骤使用阿里云公共仓库。私有仓库只有在平台支持配置相应 Registry 只读凭证时才使用。

2. 应用必须满足的运行规范

项目	要求与建议
部署模式	界面选择“模式 B · Cloudflare 容器”；后端值为 <code>c</code> ，不是 <code>b</code>
系统与架构	构建 linux/amd64；Apple Silicon 电脑同样需要显式指定
监听地址	<code>0.0.0.0:8080</code> ；不能只监听容器内的 <code>127.0.0.1</code>
网页入口	<code>GET</code> / 能打开用户界面；网页与业务 API 优先同源
健康检查	<code>GET /health</code> 快速返回 HTTP 200 JSON，例如 <code>{"ok":true}</code> ；不调用模型
进程与权限	非 root 运行；保持主进程前台运行；不依赖特权或 Docker Socket
退出处理	正确处理 SIGTERM，在有限时间内退出；取消未完成的模型请求
数据保存	容器磁盘和内存按临时数据处理；无后端持久化时，浏览器 <code>localStorage</code> 只能暂存非敏感、可丢失的前端状态，详见下方说明
网络请求	设置连接与响应超时；浏览器停止生成或断开时关闭上游流
日志与错误	记录请求编号、阶段、耗时和状态；不记录 Key、会话完整地址或原始敏感输入

前端临时存储： 未配置后端持久化服务时，前端可将少量非敏感、仅属于当前用户本机的状态（如显示偏好或可丢弃的草稿）暂存到浏览器 `localStorage`。它只能作为便捷缓存：数据受当前浏览器配置文件和来源域限制，可能被用户或浏览器清除，也不能可靠地跨设备、浏览器或用户同步。禁止存放 API Key、Authorization、`session` 参数、登录凭据、敏感对话内容或必须恢复的数据。需要长期、共享或服务端保存的数据必须使用明确配置的外部持久化服务。

8080 是本文和示例项目采用的端口。平台规格允许的其他非特权端口也必须与应用监听、镜像声明、平台表单一致；初次接入建议保持 8080。

推荐的项目结构：

```
TEXT
my-agent/
├── backend/
│   ├── app/           # Python 后端与 Agent 逻辑
│   └── requirements.txt # 锁定生产依赖
├── frontend/
│   ├── src/           # Vue 页面
│   ├── package.json
│   └── package-lock.json
├── test/               # 本地测试与模拟网关
├── Dockerfile
├── .dockerignore
├── .env.example        # 仅配置名称和占位值
├── agent-manifest.json
└── README.md
```

可以使用其他技术栈，只要遵守相同的 HTTP、凭据和生命周期规范。Python + Vue 推荐多阶段构建：先生成 Vue 静态文件，再将后端、生产依赖和静态文件放入一个运行镜像。

`.dockerignore` 至少排除下列内容；`.gitignore` 也应排除真实 `.env`：

```
TEXT
.git
.idea
.env
.env.*
!.env.example
**/node_modules
**/_pypcache_
**/.pytest_cache
**/.venv
**/*.log
**/*.pyc
```

正式发布前扫描最终镜像，而不只是扫描源代码。删除容器中的某个文件，不代表密钥已经从之前的镜像层中消失；密钥从一开始就不应进入构建上下文。

3. 模型接口与平台 Key 的接入方式

后端读取这四个变量

变量	含义	使用方式
UNEX_API_KEY	当前用户与 Agent 的托管 Key	已有 <code>sk-</code> 前缀，作为 Bearer 凭据原样使用
UNEX_API_BASE_URL	与托管 Key 配套的模型接口根地址	已含 <code>/v1</code> ，SDK 或请求代码追加具体端点
UNEX_AGENT_ID	当前 Agent 标识	用于业务隔离；不要要求用户手工填写
UNEX_USER_ID	当前使用者标识	用于业务隔离；不要信任前端自行提交的用户 ID

自定义环境变量区域不要填写 `UNEX_*`。这些变量由平台在启动实例时注入；本地测试时才由开发者模拟注入。

以下 Python 片段只演示后端读取与请求参数组织，不是完整应用：

```
PYTHON
import os

api_key = os.environ.get("UNEX_API_KEY", "")
base_url = os.environ.get("UNEX_API_BASE_URL", "").rstrip("/")
model = os.environ.get("LLM_MODEL", "claude-opus-5")

configured = bool(api_key.strip() and base_url)
# 仅在聊天入口检查 configured；缺少 Key 不应让 /health 失败。
endpoint = f"{base_url}/chat/completions" if base_url else ""
headers = {"Authorization": f"Bearer {api_key}"}
payload = {"model": model, "messages": [], "stream": True}
```

不要再给 `base_url` 拼一次 `/v1`；不要给 Key 再加一次 `sk-`；使用 SDK 时也不要把 Bearer 加进 `api_key` 参数。`messages` 应由后端校验后填入，生产请求还必须设置超时与取消处理。

Key 与 Base URL 必须成对使用

普通开发者应默认使用平台注入的 `UNEX_API_BASE_URL`，不要把某个测试域名或正式域名写死成所有环境通用的地址。

实际接入验证表明，托管 Key 与同一实例注入的 Base URL 配对时可以正常调用；手工替换成其他域名会返回 HTTP 401。这说明注入的 Key 与 URL 必须成对使用，不能据此为所有环境固定或推荐某个域名。

如果应用提供 Base URL 设置，只允许选择服务器明确配置的可信地址。切换地址前确认该接口接受当前平台 Key；不要让任意浏览器输入的地址直接接收托管 Key。普通终端用户无需填写 API Key。

模型名称也要保持一致

同时核对应用实际发送的模型名、`LLM_MODEL` 环境变量、Agent 的模型允许列表和当前账号可用渠道。前端模型名称应由后端配置返回，避免页面显示一个模型而后端调用另一个模型。

本例设置：

```
TEXT
LLM_MODEL=claude-opus-5
```

不同网关对输出长度参数和工具调用的兼容性可能不同。遇到 400 时先核对具体错误和网关支持的参数，不要在失败后自动更换模型或反复重试收费请求。

4. 页面与流式接口要注意什么

让每次请求进入同一实例

当前平台通过入口 URL 的 `session` 参数路由到用户容器。浏览器不会自动把首页的查询参数复制到 JS、CSS、图片或 API 请求中。首页能返回 200，并不代表其他资源能加载。

Python + Vue 示例采用：

1. 生产构建将 JS、CSS 和小图标内联进首页；例如 Vue + Vite 可使用 `vite-plugin-singlefile`。
2. API 使用同源相对地址，并显式携带当前 `session`。
3. 保留入口的路径前缀；页面设置 `Referrer-Policy: no-referrer`，不把 `session` 发往外部站点或日志。

下面是同源 API 地址的组织方式：

```
TYPESCRIPT
function apiUri(path: "config" | "chat"): URL {
  const url = new URL(`api/${path}`, document.baseURI);
  const session = new URL(window.location.href)
    .searchParams.get("session");
  if (session && url.origin === window.location.origin) {
    url.searchParams.set("session", session);
  }
  return url;
}
```

仅在构造同源请求时从当前入口 URL 读取 `session`；不要把它复制到 `localStorage` 或 `sessionStorage`。

若保留独立静态资源，必须验证它们的请求也满足平台路由要求，包括字体、图片和动态加载模块。仅设置 Vite 的 `base: "/"`，不能解决查询参数丢失的问题。

`/health` 在容器内部直接访问；浏览器从公开 Worker 域名访问时仍受平台会话路由约束。普通用户应从平台“打开实例”进入，不应自行去掉 `session` 后访问裸域名。

HTTP 200 不等于模型已成功

流式接口通常先返回 HTTP 200，之后才收到模型结果。以前端与后端约定的终止事件判断完成，示例协议如下：

```
TEXT
event: token
data: {"content":"OK"}

event: done
data: {"model":"claude-opus-5"}
```

失败时可能仍是 HTTP 200，但响应正文包含：

```
TEXT
event: error

data: {"code":"provider_401","retryable":false}
```

`token`、`done`、`error` 是本示例应用的 SSE 事件约定，不是所有 OpenAI 兼容接口的统一事件名。自己的前后端应约定并正确处理成功、失败、取消和流意外中断。

5. 构建镜像并在本机验证

所有命令在项目根目录执行，示例使用 macOS / Linux 的 Bash 或 zsh。

构建 amd64 镜像

```
BASH
docker buildx build \
  --platform linux/amd64 \
  --provenance=false \
  --load \
  -t my-agent:1.0.0 .

docker image inspect my-agent:1.0.0 \
  --format '{{.Os}}/{{.Architecture}} user={{.Config.User}}'
```

应看到 `linux/amd64`，运行用户应为非 `root`。设置了 `EXPOSE 8080` 只是在镜像中声明端口，应用仍必须真正监听 8080。

先检查启动和网页

```
BASH
docker run --rm -d \
  --name my-agent-local \
  --platform linux/amd64 \
  -p 127.0.0.1:8080:8080 \
  my-agent:1.0.0

curl --fail http://127.0.0.1:8080/health
curl --fail -o /dev/null http://127.0.0.1:8080/
docker logs --tail 80 my-agent-local
docker stats --no-stream my-agent-local
```

用浏览器打开 <http://127.0.0.1:8080/>。未注入模型凭据时，网页和健康检查应可用；对话应明确提示缺少凭据。日志中不应出现密钥或完整输入。

再检查一次完整对话

优先使用项目内的模拟模型网关，验证请求路径、Key 传递、模型名、流式输出与取消，不产生真实模型费用。若使用本次 SOL 示例源码，可在单独终端运行：

```
BASH
python3 test/mock_gateway.py
```

停止前一个测试容器，再模拟平台注入运行：

```
BASH
docker stop --time 15 my-agent-local

docker run --rm -d \
  --name my-agent-local \
  --platform linux/amd64 \
  -p 127.0.0.1:8080:8080 \
  -e API_BASE_URL= \
  -e UNEX_API_KEY=sk-test \
  -e UNEX_API_BASE_URL=http://host.docker.internal:3000/v1 \
  -e UNEX_AGENT_ID=local-agent \
  -e UNEX_USER_ID=local-user \
  my-agent:1.0.0
```

这段模拟命令适用于包含 `test/mock_gateway.py` 的示例项目和 Docker Desktop。`sk-test` 只能与这个本地模拟网关配合使用，不是真实模型 Key。Linux Docker Engine 需要另行配置容器访问宿主机的路径；不要把本机的 `127.0.0.1` 当成容器外的宿主机。

最后在生成中点击“停止”，确认上游调用结束；执行 `docker stop --time 15 my-agent-local`，确认进程能正常退出。页面、健康检查和模型对话分别验证，不能用其中一项替代另外两项。

6. 推送到阿里云公共镜像仓库

创建仓库并确认地址

在阿里云容器镜像服务控制台创建命名空间和仓库，选择公共访问方式。不同地域或实例类型的域名不同，以控制台提供的“仓库地址”和登录说明为准。本文不要求固定的命名空间前缀。

下面以杭州地域地址为例。把三个 `YOUR_...` 替换为你的信息；版本标签每次发布递增，不要一直覆盖同一个 `latest`。

```
BASH

REGISTRY='registry.cn-hangzhou.aliyuncs.com'

NAMESPACE='YOUR_NAMESPACE'
REPOSITORY='YOUR_REPOSITORY'
VERSION='1.0.0'
IMAGE="$REGISTRY/$NAMESPACE/$REPOSITORY:$VERSION"

docker login "$REGISTRY" --username 'YOUR_LOGIN_NAME'
docker tag my-agent:1.0.0 "$IMAGE"
docker push "$IMAGE"
```

密码仅在 Docker 的交互提示中输入，不写入脚本、Dockerfile、源码或聊天记录。这里上传的是镜像，不是直接上传前端源码 ZIP。

记录 digest，确认公共拉取

Push 完成后记录输出中的 `sha256:...`。Tag 是方便填写的名称；digest 是该次镜像内容的标识。后续平台绑定、部署、回滚都应核对 digest。

本机已登录仓库，直接 `docker pull` 成功并不能证明其他人能匿名拉取。可用一个临时的空 Docker 配置，只读取远端 manifest：

```
BASH

CHECK_CONFIG=$(mktemp -d)

docker --config "$CHECK_CONFIG" manifest inspect "$IMAGE"
rm -r "$CHECK_CONFIG"
```

匿名读取失败时先检查仓库是否公开、地址与标签是否正确。若镜像必须私有，使用平台支持的只读 Registry 凭证，不要把仓库账号密码放入 Agent 环境变量。

需要交付 TAR 时

```
BASH

docker image save -o my-agent-1.0.0-amd64.tar my-agent:1.0.0

docker load -i my-agent-1.0.0-amd64.tar
```

这是标准 Docker 镜像归档。不要使用 `docker export` 代替 `docker save`，前者导出的是容器文件系统，不是带镜像配置的同类交付物。TAR 文件 SHA-256 与 Registry 镜像 digest 是不同对象的摘要，不能相互替代。

7. 在平台填写部署配置并发布

进入自己创建的 Agent，打开开发者部署页。具体按钮名称可能随平台版本调整，按以下顺序核对状态。

准备 manifest 和表单

在项目根目录提供 `agent-manifest.json`，内容按实际能力填写：

```
JSON

{
```

```
"schema_version": 1,
"deploy_mode": "c",
"runtime": {
  "port": 8080,
  "health_path": "/health",
  "command": "",
  "instance_type": "basic",
  "idle_minutes": 15
},
"capabilities": {
  "input_types": ["text"],
  "output_types": ["text"],
  "needs_persistent_storage": false
}
}
```

它是部署信息草稿，不会替代平台校验。若平台没有自动导入功能，手工填写对应字段即可。

本例将 `needs_persistent_storage` 设为 `false`，因为应用不依赖需要长期保存的服务端数据。可选的浏览器 `localStorage` 不属于平台持久化能力，也不能代替服务端持久化需求。

表单项目	本文示例填写值
部署模式	模式 B · Cloudflare 容器
镜像来源	外部镜像仓库
镜像地址	完整的 Registry/命名空间/仓库:版本，不带 https://
入口端口	8080
健康检查	/health
启动命令	留空，使用镜像的 ENTRYPOINT / CMD
实例规格	basic 为本例；按自己应用的内存、CPU 和平台可选项确定
空闲回收	本例 15 分钟；按平台允许范围和业务需求填写
非敏感环境变量	例如 LLM_MODEL=claude-opus-5
模型允许列表	包含 claude-opus-5 或你的实际模型标识

不要往普通环境变量表单填写 Key。部分平台版本会明文保存并返回这些配置，它不是专用密钥服务。

按顺序完成发布

- 1. 保存部署模式和运行配置，提交完整镜像地址。
- 2. 执行镜像拉取与校验，核对平台显示的架构和 digest。
- 3. 等待扫描通过和镜像搬运完成；失败时先查看对应阶段的错误。
- 4. 部署运行时，等待平台显示可用状态。Worker 部署成功不等于镜像已经成功绑定。
- 5. 在开发者自测入口启动实例，从平台打开页面；查看首页、健康状态，并发送一个短问题。
- 6. 检查真实响应完成、模型正确、无 Key 泄漏，再按平台流程提交审核和发布。
- 7. 发布后从普通用户入口验证一次使用流程，确认权限和费用归属符合平台规则。

审核中或已发布的版本不要原地替换。需要改动时创建新候选版本；已有审核流程按平台规则撤回或等待处理，不绕过状态校验。

普通用户打开时应看到什么

用户从平台启动自己的实例，等待冷启动完成后进入 Agent 页面，直接使用业务功能。应用不要求用户提供 API Key；使用结束后可以从平台停止实例。容器运行费与模型调用费是两个项目，费用、最长时长和退款规则以平台当时展示的规则为准。

8. 更新、回滚和示例项目特别说明

每次更新使用新版本，例如 1.0.1。完成构建、测试和 Push 后，在平台提交新镜像引用，重新校验 digest、扫描、部署，并停止旧实例、重新启动。只推送阿里云标签不会自动更新平台已经绑定的镜像，也不会自动替换正在运行的容器。

回滚时通过平台允许的版本切换流程选择上一已验证的 digest 和对应运行配置。只改 Tag 名称或在本机重打标签，不能完成线上回滚。

如果直接使用 SOL Agent 示例

SOL Agent 1.0.5 应配置为使用平台注入的 UNEX_API_BASE_URL；仅在模型可用时保留 claude-opus-5。镜像标签为 registry.cn-hangzhou.aliyuncs.com/xxgc_lbs/study:sol-agent-mode-b-1.0.5。平台或浏览器若保存过旧的 API_BASE_URL 覆盖项，应清除或替换为当前注入值。更新镜像后仍需完成拉取、绑定、部署和实例重启。

以下保留 1.0.4 的升级说明。下载版 PDF 与当前指南同步，同时包含 1.0.5 当前配置和 1.0.4 升级说明。

本指南的通用原则是“平台注入的 Key 与 URL 配对使用”。已交付的 SOL Agent 1.0.4 镜像存在一项历史配置：它预置了 API_BASE_URL=https://api.unexhub.ai/v1，优先于平台注入的地址。

使用该示例时，应按部署环境处理：

- 平台支持保存空字符串环境变量时，将 API_BASE_URL 显式设置为空，让后端使用 UNEX_API_BASE_URL；仅删除平台表单字段，不会清除镜像内已经设置的 ENV 默认值。
- 平台不保留空值时，把 API_BASE_URL 设置为管理员确认的、与当前托管 Key 配套的地址，或者在自己的 Dockerfile 中移除该覆盖项后重新构建。
- 若浏览器曾保存过其他 Base URL，还需到“工作空间设置 → 平台托管连接”选择正确地址并保存。服务器默认值变化不一定覆盖浏览器已有偏好。
- 本次验证使用平台注入值成功；每个环境都应重新确认当前注入值。

API_BASE_URL、ALLOWED_API_BASE_URLS、LLM_MODEL 是这个示例应用支持的配置名称，不是每个 Agent 都必须实现的平台保留变量。

9. 常见问题排查表

现象	优先检查	处理方式
本地能运行，平台启动失败	镜像架构、监听地址、端口、启动命令	确认 amd64、0.0.0.0:8080；使用镜像默认命令；查看启动日志
首页空白，JS / CSS 返回 400	静态资源是否丢失 session	内联资源或满足平台路由要求；检查是否还在运行旧 digest
missing_session	是否直接访问裸域名或丢失入口参数	从平台重新打开实例；修正同源请求的 session 传递
Worker 已部署但镜像绑定失败	部署模式、审核状态、镜像状态	确认模式 B 实际值为 c；按平台流程处理审核；无需盲目重打镜像
HTTP 401 / provider_401	Key 与 Base URL 是否配套、令牌是否有效	先对照平台注入地址；必要时重启获取当前凭据，并让管理员检查鉴权日志
HTTP 402 / insufficient_user_quota	当前使用者的钱包余额	返回平台充值或按余额规则处理；停止自动重试
HTTP 403	是否为明确的 Agent 禁用码，或模型、分组限制	只有 AGENT_ACCESS_DISABLED 才按 Agent 禁用处理；其他情况检查权限
HTTP 503 / model_not_found	当前用户分组、模型允许列表、可用渠道	让平台管理员检查模型路由；上游单独可用不代表此用户的令牌路由可用
HTTP 400	请求参数、模型名、工具调用兼容性	查看安全错误码；按网关文档调整参数；必要时关闭工具做一次对比
HTTP 429 / 超时	速率、容量、超时设置	提示稍后重试，限制重试次数；取消时停止上游请求
网络面板是 200，但页面报错	SSE 正文是否含 error，是否出现完成事件	查看响应正文；不要只看 HTTP 状态
换镜像后仍显示旧模型或旧页面	平台绑定 digest、旧实例、环境变量覆盖	更新版本并重启；清理旧 LLM_MODEL 覆盖；检查后端返回的版本和模型

提交问题时提供：Agent ID、应用版本、镜像 digest、发生时间和时区、请求编号、HTTP 状态、脱敏错误码、可重现步骤。不要在公开工单中粘贴 API Key、Authorization、完整 session URL 或他人的聊天内容。

10. 发布前检查与可复制的开发需求

发布前逐项确认

- ☐ 镜像为 linux/amd64，非 root 运行，监听 0.0.0.0:8080。
- ☐ 根页面和 /health 正常；缺少模型凭据时有明确提示。
- ☐ Key 只在后端使用，镜像层、前端响应和日志均不含 Key。
- ☐ 默认模型、Agent 允许列表和当前用户可用渠道一致。
- ☐ Key 与 Base URL 来自同一套受信任的运行环境。
- ☐ 已验证携带 session 的首页、静态资源和 API 请求。
- ☐ 已完成一次流式回答、一次取消和一次异常提示验证。
- ☐ SIGTERM 后能在有限时间退出；持久数据不依赖容器临时磁盘或浏览器 localStorage。
- ☐ 如使用浏览器 localStorage，其中只有非敏感且可丢弃的界面状态；清除后不影响 Agent 基本功能。
- ☐ 仓库可被平台拉取，digest 已记录，镜像扫描通过。
- ☐ 平台运行时可用，新实例自测通过后再提交审核发布。

交给 AI 或开发人员的需求模板

TEXT

请开发一个能够部署到平台“模式 B · Cloudflare 容器”的 Agent。

名称：[填写]
用户与用途：[填写]
核心流程：[填写 3-6 个步骤]
输入与输出：[填写]
默认模型：[填写平台支持且允许调用的模型]
技术栈：Python FastAPI + Vue，或已有项目技术栈

交付要求：

1. 单容器，linux/amd64，非 root，监听 0.0.0.0:8080。

2. GET / 提供网页，GET /health 快速返回 200 JSON，不调用模型。

3. 后端读取 UNEX_API_KEY 和 UNEX_API_BASE_URL，成对使用。
Key 已有 sk-，Base URL 已含 /v1，禁止重复拼接。

4. 普通用户不填写 Key；浏览器和日志不能出现 Key。

5. 适配平台 session 路由，API 保留当前 session 和路径前缀。

6. 页面支持流式结果、错误提示、取消、复制和移动端。

7. 设置超时，断开时取消上游请求；正确处理 SIGTERM。

8. 容器磁盘为临时数据；长期、共享或服务端数据使用外部持久化。
localStorage 仅可保存非敏感、可丢弃的界面状态；严禁保存 Key、session、凭据、敏感内容或必须保留的记录。

9. 提供 Dockerfile、.dockerignore、agent-manifest.json、README，
以及本地模拟网关或等效测试。

10. 提供构建、运行、阿里云 tag/push、docker save 命令。

11. 提供平台表单填写值及测试结果；不能把模拟调用当成真实模型验证。

上传流程使用外部 Registry，不假设平台已经支持浏览器 TAR 直传。

参考与适用范围

本指南依据平台提供的《模式 B（Cloudflare Containers）Agent 容器生成、上传、使用与平台计费规格》（2026-09-23），以及 SOL Agent 1.0.4 的构建、阿里云推送和线上连通性验证整理。平台规格中的规划功能没有写成已上线功能；平台界面、仓库要求和权限以实际开放能力为准。

- [Cloudflare Containers 架构与生命周期](#)
- [Cloudflare Containers 镜像管理](#)
- [Cloudflare Containers 规格与限制](#)

Cloudflare 文档用于理解底层容器能力；平台开发者仍通过平台部署入口操作，不需要照抄 Cloudflare 管理命令或自行提供管理 Token。